LEMBAR KERJA MANDIRI DAN BANK SOAL

Pemrograman untuk Ilmu Data dan Komputasi

Mata Kuliah : MA25-21008, 3 SKS
Program Studi : Matematika, Fakultas Sains, ITERA
Semester : Ganjil 2026/2027
Cakupan UTS : M1-M7
Dosen : Rifky Fauzi; Triyana Muliawati, S.Si., M.Si.

Nama ..... NIM .....
Kelas ..... Tanggal .....

Tujuan dan penggunaan

Lembar kerja mandiri ini disusun sebagai bank latihan resmi untuk materi M1-M7. Asisten praktikum akan memilih soal pre-test dari bank soal ini. Angka, data, atau konteks boleh diubah menyesuaikan kebutuhan latihan karena kompetensi yang diuji harus tetap sama. Soal juga dapat digunakan sebagai bahan latihan dan *mockup* UTS (kisi-kisi). Mahasiswa diharapkan menuliskan prediksi atau analisis terlebih dahulu, kemudian mencoba kode untuk memeriksa hasilnya.

Peta materi dan sumber utama

M	Fokus	Rujukan utama
M1	Tipe data, variabel, operasi, I/O, kondisional, pengulangan	Miller-Ranum Ch. 1; Pine Ch. 2 dan 5; McKinney Ch. 2
M2	Fungsi, parameter, return, tuple/unpacking, scope, fungsi sebagai objek, lambda	Pine Ch. 7; McKinney Ch. 3
M3	Representasi algoritma, list, dictionary, array NumPy, comprehension	Miller-Ranum Ch. 1; Pine Ch. 3 dan 5; McKinney Ch. 3–4
M4	Exception handling, debugging, testing, terminasi, rekursi, Newton	Miller-Ranum Ch. 4; Pine Ch. 2 dan 7; McKinney Ch. 3 dan Appendix B
M5	Analisis algoritma dan kompleksitas Big-O	Miller-Ranum Ch. 2
M6	Searching, sorting, tracing algoritma, benchmark	Miller-Ranum Ch. 5
M7	Vektor, matriks, transpose, invers, SPL, norma, nilai eigen, data sebagai matriks	Pine Ch. 9; McKinney Ch. 4

Sumber utama:

1. Brad Miller dan David Ranum, *Problem Solving with Algorithms and Data Structures Using Python*, Release 3.0.
2. David J. Pine, *Introduction to Python for Science and Engineering*, CRC Press, 2019.
3. Wes McKinney, *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*, 3rd ed., O'Reilly, 2022.

Catatan: beberapa soal diadaptasi dari ide, contoh, self-check, dan latihan pada tiga sumber di atas. Soal dirancang ulang untuk konteks mahasiswa Matematika tahun kedua.

Cakupan konsep inti muncul pada kuis, tugas atau UTS

No.	Konsep	Latihan terkait
1	Tipe data dan konversi tipe	M1.1
2	Tuple dan unpacking	M2.3
3	Dictionary	M3.3, M3.8
4	`None` dan data tidak tersedia	M1.7
5	`break` dan `continue`	M1.7
6	Loop bersarang	M1.8, M3.6
7	Multiple return	M2.3
8	Fungsi sebagai objek	M2.5, M2.7
9	Lambda dan bentuk Pythonic	M2.6
10	Exception handling	M4.2
11	Debugging	M4.3–M4.5
12	`print`, `return`, dan `None`	M2.1
13	Testing dengan `assert`	M4.6
14	Terminasi program	M4.7
15	Rekursi	M4.8–M4.10
16	Kompleksitas algoritma	M5.1–M5.10
17	Searching	M6.1–M6.2, M6.6
18	Sorting	M6.3–M6.8
19	Benchmark implementasi	M6.9–M6.10
20	Data sebagai matriks	M7.10

M1: Python Dasar dan Alur Logika

Konsep algoritma, tipe data, variabel, operasi, input/output, kondisional, dan pengulangan

M1.1: Prediksi tipe data dan hasil L1

Tanpa menjalankan Python terlebih dahulu, tentukan apakah setiap ekspresi menghasilkan nilai, menghasilkan `True/False`, atau menimbulkan error. Jika menghasilkan nilai, tuliskan nilai dan tipenya. Setelah itu verifikasi dengan Python.

```
1 + True
True + True
'1' + '0'
'1' + 0
int('1') + 0
3 / 2
3 // 2
2**3.0
bool(0), bool(-2), bool('0')
```

Jelaskan mengapa `1 + True` dan `'1' + 0` berperilaku berbeda.

M1.2: Bilangan dan representasi L1

Tuliskan program yang menerima bilangan bulat positif n dan mencetak: nilai binernya. Uji untuk $n = 13$, 31, dan 255. Jelaskan hubungan banyak bit minimum dengan nilai n. Kemudian buat program untuk mengubah bilangan biner ke bilangan desimal.

M1.3: Ekspresi matematika L2

Untuk $x = 2.4$ dan $y = -1.2$, tuliskan satu ekspresi Python untuk menghitung

$$f(x, y) = \frac{e^{x-y} + \sqrt{x^2 + y^2}}{1 + |xy|}.$$

Bandingkan hasil menggunakan `math` dan NumPy. Prediksi tipe hasilnya.

M1.4: Diskriminan persamaan kuadrat L2

Buat program yang menerima a, b, c untuk $ax^2 + bx + c = 0$. Program harus menentukan apakah akar real berbeda, real kembar, atau kompleks berdasarkan $D = b^2 - 4ac$. Setelah klasifikasi, hitung akar yang sesuai. Uji minimal tiga kasus yang mewakili ketiga kondisi.

M1.5: Fungsi sesepenggal L2

Diberikan

$$f(x) = \begin{cases} x^2 + 1, & x < 0, \\ 2x + 1, & 0 \le x < 3, \\ \sqrt{x + 6}, & x \ge 3. \end{cases}$$

Tuliskan program yang menerima x dan menghitung $f(x)$. Uji tepat di sekitar batas $x = 0$ dan $x = 3$. Mengapa pengujian pada batas penting?

M1.6: Pengulangan pada list L2

Diberikan `data = [3, -2, 0, 7, -5, 4, 9]`. Dengan `for`, hitung: jumlah elemen positif, jumlah kuadrat elemen negatif, dan rata-rata semua elemen nonnol. Jangan gunakan NumPy.

M1.7: break dan continue L3

Diberikan list

```
data = [12, 7, None, 8, -3, 0, 11, 5]
```

Tulis loop yang mengabaikan `None` dengan `continue`, berhenti ketika menemukan 0 dengan `break`, lalu menghitung jumlah nilai yang sempat diproses. Prediksi output sebelum menjalankan kode.

M1.8: Segitiga bintang dan loop bersarang L3

Untuk bilangan bulat $n \geq 1$, buat keluaran

```
*
**
***
...
```

hingga n baris menggunakan loop bersarang. Setelah itu modifikasi menjadi segitiga siku-siku terbalik. Berapa banyak karakter `*` yang dicetak sebagai fungsi dari n? Nyatakan dengan rumus.

M1.9: Faktor dan bilangan prima L4

Buat program yang menerima $n > 1$ dan mencari faktor positif terkecil selain 1 menggunakan `while`. Gunakan faktor tersebut untuk menentukan apakah n prima. Jelaskan mengapa loop pasti berhenti. Uji $n = 101$, 221, dan 997.

M1.10: Pendekatan numerik ke π L4

Keliling poligon beraturan n sisi yang terinskripsi pada lingkaran berjari-jari $1/2$ adalah

$$p_n = n \sin\left(\frac{\pi}{n}\right).$$

Hitung p_n untuk $n = 3, 4, 10, 100, 10^4, 10^6$. Amati kecenderungannya. Tulis loop dan jelaskan apa yang terjadi secara matematis ketika n membesar.

M2: Fungsi, Scope, dan Gaya Pythonic

Subprogram, argumen, return, tuple/unpacking, fungsi sebagai objek, fungsi lambda, dan modularisasi

M2.1: Empat bentuk fungsi dan makna return L1

Buat empat fungsi berikut:

1. tanpa argumen dan tanpa `return`;
2. tanpa argumen tetapi memiliki `return`;
3. memiliki dua argumen dan tanpa `return`;
4. memiliki tiga argumen dan memiliki `return`.

Gunakan konteks matematika sederhana. Untuk setiap fungsi, prediksi nilai yang dikembalikan Python. Jelaskan perbedaan antara menampilkan nilai dengan `print` dan mengembalikan nilai dengan `return`.

M2.2: Argumen posisi, keyword, dan nilai default L2

Definisikan

```
def kuadrat(x, a=1.0, b=0.0, c=0.0):
    return a*x**2 + b*x + c
```

Hitung hasil beberapa pemanggilan dengan argumen posisi dan keyword. Buat satu contoh pemanggilan yang salah karena urutan atau nama argumen. Jelaskan kapan nilai default membantu membuat fungsi lebih mudah digunakan.

M2.3: Multiple return, tuple, dan unpacking L2

Tulis fungsi `stat_ringkas(a,b,c,d)` yang mengembalikan empat nilai sekaligus: minimum, maksimum, rata-rata, dan rentang. Gunakan

```
minimum, maksimum, rata, rentang = stat_ringkas(a,b,c,d)
```

untuk unpacking. Tunjukkan tipe objek yang sebenarnya dikembalikan fungsi. Apa yang terjadi jika hasil empat elemen tersebut ditampung hanya ke dua nama variabel saja?

M2.4: Variabel lokal dan global L2

Prediksi output sebelum menjalankan program.

```
x = 10

def f(a):
    x = a + 2
    return x

print(f(5))
print(x)
```

Buat versi kedua yang menggunakan `global x`. Bandingkan kedua desain. Jelaskan mengapa fungsi yang mengubah variabel global biasanya lebih sulit diuji.

M2.5: Fungsi sebagai objek dan sebagai argumen L3

Gunakan pola berikut.

```
def proses(x, f):
    return f(x)

def kuadrat(x):
    return x**2
```

Tambahkan fungsi `kubik` dan `tambah1`. Terapkan fungsi-fungsi tersebut pada nilai yang sama melalui `proses`. Kemudian simpan objek fungsi ke dalam sebuah list dan panggil semuanya di dalam loop. Bedakan `kuadrat(5)` dengan objek fungsi `kuadrat`.

M2.6: Lambda L3

Definisikan

$$f(x) = 3x^2 - 2x + 1$$

dengan `lambda`. Evaluasi pada $x = -3, -2, \ldots, 3$. Tulis juga versi menggunakan fungsi bernama `def`. Bandingkan keterbacaan keduanya dan berikan satu contoh keadaan ketika lambda sebaiknya tidak digunakan.

M2.7: Komposisi fungsi L3

Diberikan $f(x) = x^2 + 1$ dan $g(x) = 2x - 3$. Tulis

```
def komposisi(f, g, x):
    ...
```

untuk menghitung $(f \circ g)(x)$. Hitung juga $(g \circ f)(x)$ untuk beberapa nilai x dan tunjukkan bahwa komposisi fungsi tidak selalu komutatif.

M2.8: Fungsi matematika dengan pemeriksaan domain L4

Tulis fungsi untuk

$$h(x) = \frac{\sqrt{x+2}}{x-1}.$$

Sebelum melakukan perhitungan, fungsi harus

memeriksa domain dan mengembalikan `None` jika $x < -2$ atau $x = 1$. Uji pada nilai batas dan nilai di luar domain. Pada minggu berikutnya, fungsi semacam ini akan diperbaiki lagi menggunakan exception handling.

M3: Representasi Algoritma, Collection, dan Array NumPy

Pseudocode, list, dictionary, comprehension, nested iteration, dan komputasi array

Template pseudocode yang digunakan

```
ALGORITMA NamaAlgoritma
INPUT:  data masukan dan syaratnya
OUTPUT: keluaran yang diharapkan
BEGIN
    inisialisasi
    langkah-langkah
    IF kondisi THEN
        ...
    ELSE
        ...
    ENDIF
    FOR / WHILE ...
        ...
    ENDFOR / ENDWHILE
    RETURN hasil
END
```

Pseudocode tidak harus mengikuti sintaks Python. Input, output, kondisi, pengulangan, dan urutan langkah harus dapat ditelusuri dengan jelas.

M3.1: Dari masalah ke pseudocode L1

Tuliskan pseudocode untuk mencari nilai maksimum dan indeks kemunculan pertamanya dari sekumpulan bilangan tanpa menggunakan `max`. Setelah itu implementasikan dalam Python dan uji pada `[4,1,9,2,9,3]`.

M3.2: Pseudocode algoritma Euclid L2

Tuliskan pseudocode algoritma Euclid untuk mencari *Great Common Divisor* (GCD) dua bilangangcd(a, b). Implementasikan dengan `while`. Gunakan untuk menghitung gcd$(252, 105)$ dan catat urutan pasangan (a, b) sampai algoritma berhenti.

M3.3: Iterasi list dan dictionary L2

Buat contoh `for` yang mengiterasi list dan dictionary. Gunakan dictionary nilai mahasiswa dengan struktur `nama:nilai`. Iterasi pasangan `key,value`, hitung rata-rata, dan tentukan nama dengan nilai tertinggi.

M3.4: List comprehension sebagai operasi vektor L2

Diberikan $x = [-2, -1, 0, 1, 2]$. Dengan list comprehension buat

$$y_i = 3x_i^2 - 2x_i + 1, \qquad z_i = x_i y_i.$$

Hitung $\sum_i z_i$. Tulis versi loop biasa dan bandingkan struktur keduanya.

M3.5: List comprehension pada matriks L3

Diberikan

```
A = [[1,2,3],
     [4,5,6],
     [7,8,9]]
```

Tanpa NumPy: ambil kolom terakhir, diagonal utama, dan bentuk list berisi dua kali kuadrat elemen kolom tengah menggunakan comprehension.

M3.6: Membentuk matriks dengan loop bersarang L3

Dengan dua `for`, bentuk matriks $A \in \mathbb{R}^{4\times 4}$ dengan

$$a_{ij} = i + j + ij, \qquad i, j = 1, 2, 3, 4.$$

Simpan sebagai list of lists. Tentukan diagonal utama dan periksa secara algoritmik apakah A simetris. Setelah itu konversi ke NumPy array dan periksa $A = A^T$.

M3.7: Tipe data pada NumPy array L2

Konversi `[1,4.,-2,7]` menjadi NumPy array dan periksa `dtype`. Jelaskan mengapa seluruh elemen menjadi satu tipe. Buat array integer dan float, kemudian bandingkan hasil pembagian dan operasi element-wise sederhana.

M3.8: Dictionary dan standardisasi L3

Diberikan

```
nilai = {'A':82, 'B':71, 'C':95, 'D':64}
```

Hitung rata-rata dan simpangan baku populasi. Bentuk dictionary baru dengan

$$z_i = \frac{x_i - \bar{x}}{s}.$$

Cetak key, nilai asli, dan skor z. Verifikasi bahwa rata-rata skor z mendekati nol.

M3.9: Loop Python dan NumPy vectorization L3

Ambil 1000 titik x_i pada $[0, 2\pi]$. Hitung

$$y_i = \sin(x_i) + x_i^2$$

dengan loop Python dan dengan operasi NumPy. Periksa bahwa hasil keduanya sama sampai toleransi tertentu. Fokus soal ini adalah kesetaraan hasil, bukan pengukuran waktu.

M3.10: Slicing dan beda hingga L4

Diberikan posisi y_i dan waktu t_i dalam dua NumPy array dengan panjang sama. Bentuk

$$v_i = \frac{y_{i+1} - y_i}{t_{i+1} - t_i}$$

tanpa loop eksplisit, hanya menggunakan slicing. Jelaskan mengapa panjang array v satu lebih kecil dari y dan t.

M4: Exception, Debugging, Testing, Terminasi, dan Rekursi

Kesalahan program, penanganan exception, assert, base case, recursive case, dan iterasi Newton

Halt atau terminasi

Suatu program atau algoritma dikatakan *halt* untuk sebuah input jika eksekusinya berhenti setelah sejumlah langkah yang berhingga. Pada loop atau rekursi, pertanyaan pentingnya adalah: apakah keadaan program bergerak menuju kondisi berhenti untuk setiap input yang diizinkan?

M4.1: Tiga jenis masalah pada program L1

Untuk setiap kasus berikut, klasifikasikan sebagai syntax error, runtime error, atau logical error. Berikan alasan dan perbaikan singkat.

```
# A
if x > 0
    print(x)

# B
x = 3 / 0

# C
def rata(data):
    return sum(data) / (len(data) + 1)
```

Tambahkan satu contoh buatan sendiri untuk masing-masing jenis kesalahan.

M4.2: Exception handling dan validasi input L2

Buat fungsi `rasio(a,b)` yang mencoba mengubah a dan b menjadi `float`, lalu menghitung a/b. Tangani secara terpisah `ValueError`, `TypeError`, dan `ZeroDivisionError`. Uji pada

```
('3.5','2'), ('abc',2), (3,0), (None,4)
```

Jelaskan mengapa `except:` tanpa jenis exception dapat menyembunyikan bug.

M4.3: Debugging return yang terlalu dini L2

Fungsi berikut dimaksudkan menghitung rata-rata.

```
def rata(data):
    total = 0
    for x in data:
        total += x
        return total / len(data)
```

Prediksi hasil untuk `[2,4,6,8]`, temukan kesalahan logikanya, lalu perbaiki. Tambahkan perlakuan untuk data kosong.

M4.4: Debugging algoritma Newton L3

Fungsi berikut merupakan iterasi Newton untuk menghitung akar kadang bermasalah.

```
def akar(n, tol=1e-10):
    x = 0.0
    while abs(x*x - n) > tol:
        x = 0.5 * (x + n/x)
    return x
```

Identifikasi penyebab kegagalan. Pilih initial guess yang lebih baik, tangani $n = 0$ dan $n < 0$, lalu uji pada beberapa nilai yang jawabannya diketahui.

M4.5: Debugging kondisi batas L3

Diberikan fungsi yang menentukan apakah bilangan bulat positif n prima dengan mencoba pembagi $2, 3, \ldots, \lfloor\sqrt{n}\rfloor$. Buat implementasi yang sengaja salah pada satu kondisi batas, tunjukkan test case yang membongkar bug tersebut, lalu perbaiki.

M4.6: Testing dengan assert L3

Tulis fungsi jarak Euclid dua vektor berdimensi sama. Gunakan `assert` untuk menguji: dua vektor sama, dimensi satu, komponen negatif, simetri $d(x, y) = d(y, x)$, dan $d((0, 0), (3, 4)) = 5$. Tambahkan pemeriksaan bahwa panjang dua vektor harus sama.

M4.7: Terminasi loop L3

Untuk setiap potongan kode, tentukan apakah loop halt untuk setiap bilangan bulat positif n.

```
# A
i = n
while i > 0:
    i = i // 2

# B
i = n
while i != 1:
    i = i - 2

# C
i = n
while i > 0:
    i = i + 1
```

Untuk B, klasifikasikan input yang halt dan yang tidak. Untuk A, berikan ukuran bilangan bulat nonnegatif yang turun menuju kondisi berhenti.

M4.8: Rekursi factorial L2

Tulis `factorial(n)` secara rekursif dan iteratif. Tandai base case dan recursive case. Telusuri call stack untuk $n = 5$. Bandingkan kebutuhan penyimpanan keadaan pada kedua versi.

M4.9: Rekursi barisan L3

Didefinisikan

$$a_0 = 2, \qquad a_n = 3a_{n-1} - 2, \quad n \geq 1.$$

Tulis fungsi rekursif, hitung a_0 sampai a_6, tebak rumus eksplisit a_n, lalu verifikasi hasil rekursif terhadap rumus tersebut.

M4.10: Rekursi dan perhitungan berulang L4

Diberikan

$$u_0 = 1, \qquad u_1 = 3, \qquad u_n = u_{n-1} + 2u_{n-2}.$$

Tulis fungsi rekursif dan gambar pohon pemanggilan untuk u_5. Tandai submasalah yang dihitung berulang. Jelaskan secara kualitatif mengapa implementasi rekursif langsung dapat menjadi mahal.

M5: Kompleksitas Algoritma Big-O

Ukuran input, jumlah operasi, orde pertumbuhan, dan time-space

trade-off

M5.1: Urutkan orde pertumbuhan L1

Urutkan dari pertumbuhan paling lambat ke paling cepat:

$$1, \quad \log n, \quad n, \quad n \log n, \quad n^2, \quad n^3, \quad 2^n, \quad n!.$$

Untuk $n = 10$ dan $n = 100$, bandingkan secara numerik beberapa orde yang masih masuk akal dihitung.

M5.2: Big-O nested loop L2

Tentukan Big-O dan jelaskan dari banyak eksekusi baris terdalam.

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

Pilihan: $O(n)$, $O(n^2)$, $O(\log n)$, $O(n^3)$.

M5.3: Dua loop berurutan L2

Tentukan Big-O.

```
test = 0
for i in range(n):
    test = test + 1
for j in range(n):
    test = test - 1
```

Mengapa dua loop $O(n)$ yang berurutan tidak menjadi $O(n^2)$?

M5.4: Loop membagi dua L2

Tentukan Big-O dan jumlah iterasi sebagai fungsi dari n.

```
i = n
while i > 0:
    k = 2 + 2
    i = i // 2
```

Hubungkan jumlah iterasi dengan $\lfloor \log_2 n \rfloor$.

M5.5: Menghitung T(n) L3

Untuk kode berikut, hitung jumlah eksekusi `s += 1`.

```
s = 0
for i in range(n):
    for j in range(i + 1):
        s += 1
```

Tunjukkan bahwa banyak operasi adalah $1 + 2 + \cdots + n$ dan simpulkan Big-O.

M5.6: Loop versus rumus tertutup L3

Bandingkan dua algoritma untuk $S(n) = 1 + 2 + \cdots + n$: satu menggunakan loop dan satu menggunakan $n(n + 1)/2$. Tentukan kompleksitas masing-masing. Rancang eksperimen waktu sederhana dan jelaskan mengapa waktu aktual saja tidak cukup untuk mendefinisikan kompleksitas.

M5.7: Kompleksitas operasi list L3

Klasifikasikan secara kualitatif terhadap panjang list n: indexing `x[i]`, `append`, `pop()`, `pop(0)`, `v in x`, dan sorting. Verifikasi minimal dua klaim dengan `timeit`.

M5.8: Time-space trade-off L4

Dua algoritma mendeteksi anagram. Algoritma A menyortir kedua string. Algoritma B menghitung frekuensi karakter dengan array penghitung. Bandingkan waktu dan memori. Untuk alfabet berukuran tetap, tentukan kompleksitas waktu Algoritma B terhadap panjang string n.

M5.9: Model biaya komputasi L4

Misalkan

$$T_1(n) = 7n^2 + 3n + 40, \qquad T_2(n) = 0.001n^3 + 100n.$$

Tentukan Big-O masing-masing. Gunakan Python untuk mencari kisaran n ketika $T_1(n) < T_2(n)$ dan sebaliknya. Jelaskan perbedaan antara pernyataan asimtotik dan keputusan praktis pada ukuran input terbatas.

M5.10: Eksperimen pertumbuhan L4

Rancang eksperimen untuk membedakan secara empiris fungsi yang kira-kira $O(n)$ dan $O(n^2)$. Pilih urutan nilai n, tentukan apa yang diukur, dan jelaskan pola rasio waktu yang diharapkan ketika n digandakan.

M6: Searching, Sorting, dan Analisis Kinerja

Tracing binary search, selection sort, merge sort, dan hubungan teori dengan benchmark

M6.1: Recursive binary search, key ditemukan L2

Diberikan list terurut

$$[3, 5, 6, 8, 11, 12, 14, 15, 17, 18].$$

Dengan recursive binary search, urutan perbandingan mana yang benar untuk mencari key 8?

1. 11, 5, 6, 8
2. 12, 6, 11, 8
3. 3, 5, 6, 8
4. 18, 12, 6, 8

Tunjukkan indeks kiri, kanan, dan tengah pada setiap langkah.

M6.2: Recursive binary search, key tidak ada L2

Gunakan list yang sama. Tentukan urutan perbandingan untuk key 16 dan jelaskan kapan algoritma menyimpulkan bahwa 16 tidak ada.

1. 11, 14, 17
2. 18, 17, 15
3. 14, 17, 15
4. 12, 17, 15

M6.3: Selection sort setelah tiga pass L3

Diberikan

$$[11, 7, 12, 14, 19, 1, 6, 18, 8, 20].$$

Lakukan selection sort secara manual. Tuliskan keadaan list setelah pass ke-1, ke-2, dan ke-3. Nyatakan dengan jelas definisi satu *pass* pada implementasi Anda.

M6.4: Trace merge sort L3

Untuk

$$[21, 1, 26, 45, 29, 28, 2, 9, 16, 49, 39, 27, 43, 34, 46, 40],$$

gambar tiga tingkat awal pohon rekursi merge sort. Setelah tiga pemanggilan rekursif pada cabang paling kiri, sublist mana yang sedang diproses?

M6.5: Merge pertama L3

Untuk list pada M6.4, tentukan dua list pertama yang digabung kembali. Tunjukkan hasil merge tersebut, lalu lanjutkan satu tahap merge berikutnya.

M6.6: Sequential versus binary search L3

Implementasikan `sequential_search` dan `binary_search` untuk list terurut. Catat banyak perbandingan untuk key di awal, tengah, akhir, dan tidak ada. Hubungkan hasil dengan $O(n)$ dan $O(\log n)$.

M6.7: Sorting dan orde pertumbuhan L3

Pasangkan algoritma berikut dengan kompleksitas worst-case: bubble sort, selection sort, insertion sort, merge sort, dan quicksort standar. Jelaskan algoritma mana pada daftar tersebut yang dijamin $O(n \log n)$ pada worst-case.

M6.8: Invariant pada selection sort L4

Nyatakan invariant sederhana untuk selection sort: setelah pass ke-k, bagian mana dari list yang sudah berada pada posisi final? Gunakan invariant tersebut untuk menjelaskan mengapa algoritma menghasilkan list terurut.

M6.9: Benchmark insertion sort dan merge sort L4

Rancang benchmark yang adil untuk membandingkan insertion sort dan merge sort. Gunakan ukuran input meningkat, data acak yang sama untuk kedua algoritma, dan pengulangan pengukuran. Laporkan median atau rata-rata waktu dan jelaskan mengapa satu pengukuran tidak cukup.

M6.10: Membaca benchmark dengan hati-hati L4

Buat tiga jenis input: acak, terurut, dan hampir terurut. Bandingkan insertion sort dan merge sort secara empiris. Jelaskan mengapa hasil benchmark dapat dipengaruhi ukuran input, konstanta implementasi, dan struktur data, walaupun Big-O tetap memberi informasi asimtotik.

M7: Aljabar Linier Komputasional

Array 1D dan 2D, operasi matriks, transpose, invers, SPL, norma, nilai eigen, dan data sebagai matriks

M7.1: Vektor sebagai list dan NumPy array L1

Diberikan $x = (1, -2, 3)$ dan $y = (4, 0, -1)$. Hitung manual $x + y$, $2x - y$, $x \cdot y$, $\|x\|_2$, dan $\|x - y\|_2$. Implementasikan dengan Python list dan NumPy array. Catat operasi yang berbeda perilakunya.

M7.2: Element-wise dan perkalian matriks L2

Diberikan

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 2 & 0 \\ 1 & 5 \end{pmatrix}.$$

Hitung manual $A \odot B$ dan AB. Implementasikan dengan NumPy. Jelaskan perbedaan `A*B` dan `A@B`.

M7.3: Perkalian matriks manual L2

Tulis `matmul_manual(A,B)` untuk list of lists menggunakan tiga loop. Fungsi harus memeriksa kesesuaian dimensi. Uji matriks 2×3 dikali 3×2 dan bandingkan dengan NumPy.

M7.4: Transpose tiga cara L2

Untuk

```
A = [[1,2,3],
     [4,5,6]]
```

buat transpose dengan loop bersarang, list comprehension, dan `np.array(A).T`. Jelaskan perubahan ukuran dari $m \times n$ menjadi $n \times m$.

M7.5: Invers matriks 2x2 L3

Tulis fungsi berdasarkan

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Fungsi harus menolak matriks singular. Bandingkan dengan `np.linalg.inv` dan verifikasi $AA^{-1} \approx I$.

M7.6: SPL dan determinan L3

Diberikan

$$A = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 4 \end{pmatrix}.$$

Periksa determinan. Jika $\det(A) \neq 0$, selesaikan $Ax = b$ dengan invers manual, `np.linalg.inv`, dan `np.linalg.solve`. Bandingkan hasil dan metode.

M7.7: Residual dan norma L3

Untuk solusi numerik $\hat{x}$ dari $Ax = b$, definisikan $r = b - A\hat{x}$. Tulis fungsi yang mengembalikan $\|r\|_2$.

Uji pada solusi benar dan pada solusi yang diganggu sebesar 10^{-3} pada satu komponen.

M7.8: SPL 3x3 L3

Selesaikan

$$\begin{aligned} 2x_1 + 4x_2 + 6x_3 &= 4, \\ x_1 - 3x_2 - 9x_3 &= -11, \\ 8x_1 + 5x_2 - 7x_3 &= 1. \end{aligned}$$

Bentuk array A dan b, gunakan `np.linalg.solve`, lalu verifikasi dengan residual. Jelaskan mengapa menyelesaikan sistem langsung umumnya lebih disukai daripada membentuk A^{-1} terlebih dahulu.

M7.9: Nilai eigen dan verifikasi L4

Untuk

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix},$$

hitung nilai eigen secara manual dari $\det(A - \lambda I) = 0$. Gunakan `np.linalg.eig`. Untuk setiap pasangan (λ, v), verifikasi numerik $Av \approx \lambda v$ menggunakan norma selisih.

M7.10: Data sebagai matriks L4

Data empat observasi dengan tiga fitur ditulis

$$X = \begin{pmatrix} 1 & 2 & 0 \\ 2 & 1 & 1 \\ 4 & 3 & 2 \\ 3 & 4 & 1 \end{pmatrix}.$$

Hitung vektor rata-rata kolom $\bar{x}$, centered matrix $X_c = X - \mathbf{1}\bar{x}^T$, dan Gram matrix $G = X_c^T X_c$. Kerjakan terlebih dahulu dengan loop/list, lalu dengan NumPy. Periksa bahwa G simetris dan jelaskan hubungan $X_c^T X_c$ dengan variasi antarfitur.